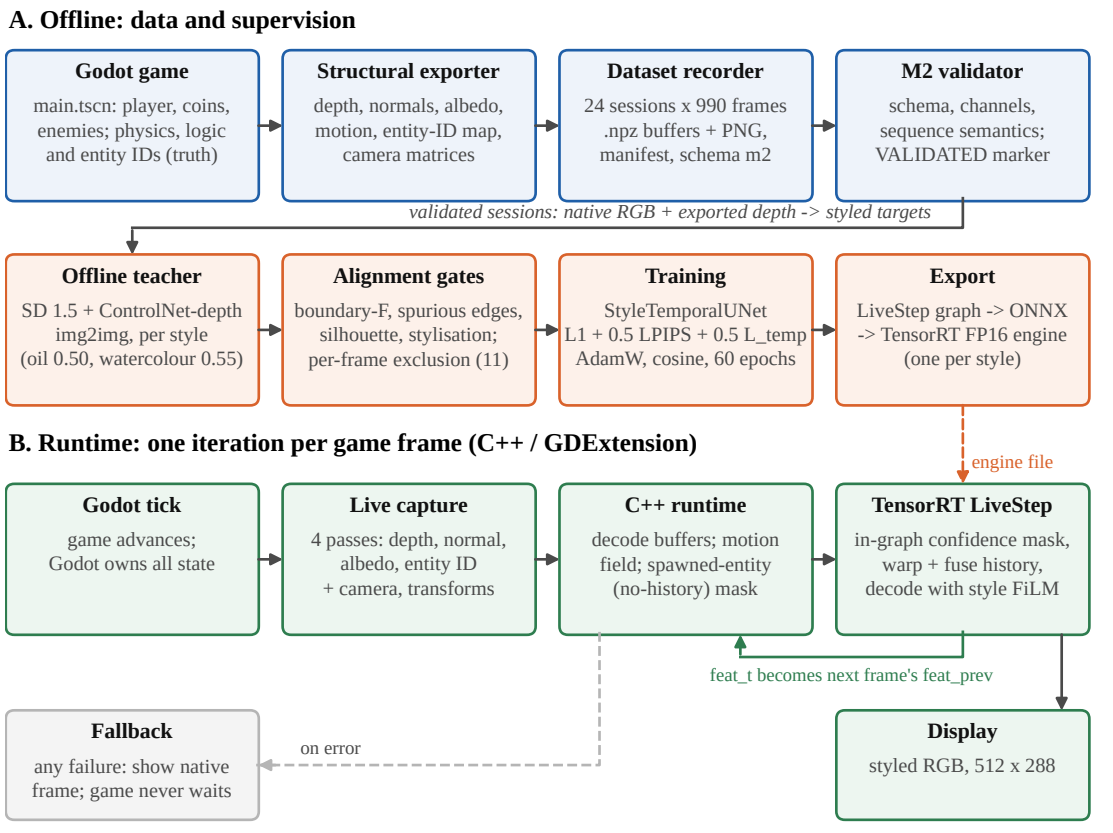
A. Offline: data and supervision
Godot game
main.tscn: player, coins, enemies; physics, logic and entity IDs (truth)
Structural exporter
depth, normals, albedo, motion, entity-ID map, camera matrices
Dataset recorder
24 sessions x 990 frames .npz buffers + PNG, manifest, schema m2
M2 validator
schema, channels, sequence semantics; VALIDATED marker
validated sessions: native RGB + exported depth -> styled targets
Offline teacher
SD 1.5 + ControlNet-depth img2img, per style (oil 0.50, watercolour 0.55)
Alignment gates
boundary-F, spurious edges, silhouette, stylisation; per-frame exclusion (11)
Training
StyleTemporalUNet L1 + 0.5 LPIPS + 0.5 L_temp AdamW, cosine, 60 epochs
Export
LiveStep graph -> ONNX -> TensorRT FP16 engine (one per style)
B. Runtime: one iteration per game frame (C++ / GDExtension)
engine file
Godot tick
game advances; Godot owns all state
Live capture
4 passes: depth, normal, albedo, entity ID + camera, transforms
C++ runtime
decode buffers; motion field; spawned-entity (no-history) mask
TensorRT LiveStep
in-graph confidence mask, warp + fuse history, decode with style FiLM
feat_t becomes next frame's feat_prev
Fallback
any failure: show native frame; game never waits
on error
Display
styled RGB, 512 x 288